REDES, WINDOWS E ACTIVE DIRECTORY / SEGURANÇA DE SISTEMAS

INVESTIGAÇÃO DE TRÁFEGO COM WIRESHARK

Perguntar, observar, correlacionar e comprovar: planejar a captura, interpretar protocolos, investigar falhas e relacionar pacotes aos logs do Windows e do domínio.

VERIFICADO NESTA EDIÇÃO

54 filtros de exibição e 5 de captura validados no TShark 4.2.2 · saídas reais de análise · buffer circular do dumpcap executado

PRÁTICA COM ARQUIVOS

10 capturas sintéticas anexadas, reproduzíveis por script e com SHA256 · casos A a D e caso integrado sobre elas

Apostila de consulta e laboratório • Outubro de 2026 • Fontes: documentação oficial do Wireshark, RFC Editor e Microsoft

As capturas anexadas foram montadas por script com endereços fictícios: não são tráfego real. Nada aqui explora vulnerabilidades. Laboratórios em rede de teste autorizada.

EDIÇÃO 2 / 29 DE OUTUBRO DE 2026 / 35 PÁGINAS

Como utilizar esta apostila

Cada investigação deve responder a uma pergunta concreta e declarar os limites do ponto de captura. O percurso começa numa consulta DNS e numa conexão TCP e termina num acesso negado a um compartilhamento, correlacionado com os logs do Windows.

Rotas de leitura

Preciso investigar...	Começar por...
Não vejo a comunicação	Capítulos 2 a 5: ponto de captura, preparação, tempo e qualidade.
Nome não resolve	Capítulo 8 (DNS) e captura `01_dns`.
Conexão não abre	Capítulos 9 e 10 e capturas `03` e `04`.
Aplicação lenta	Capítulos 11 e 12, caso A (capítulo 21).
HTTPS não mostra conteúdo	Capítulo 13.
Falha no domínio ou no compartilhamento	Capítulos 14 e 15, caso integrado (capítulo 22).
Quero praticar	Capítulos 16 a 20 e as capturas anexadas.
Preciso documentar	Apêndices A a C.

Convenções

- `EXECUTADO` marca saídas reais desta edição: TShark 4.2.2 e dumpcap em Linux sobre as capturas sintéticas, e uma captura real na interface de loopback (capítulo 19).
- `NÃO EXECUTADO` marca o que depende de Windows, Npcap ou domínio: os laboratórios dos capítulos 17 e 18 e os scripts PowerShell (sintaxe validada).
- `SINTÉTICO` marca capturas e cenários montados para estudo. As capturas sintéticas são arquivos pcapng legítimos, mas cada pacote foi construído por script; tempos e falhas foram escolhidos, não observados.
- `CORREÇÃO` marca pontos corrigidos em relação à Edição 1 (lista no Apêndice E).
- Filtros de exibição aparecem em fonte monoespaçada. Todos os citados foram validados no TShark 4.2.2 (Apêndice B). Nomes de campo e menus podem mudar entre versões: confira na sua.

Anexo

O PDF incorpora `laboratorio_wireshark.zip` com as 10 capturas, o script que as gera, as análises em shell e PowerShell, a lista de filtros e o validador. Extraia pelo painel de anexos do leitor de PDF ou com `pdfdetach -saveall`. Abra as capturas diretamente no Wireshark.

Sumário

FUNDAMENTOS

01 Método de investigação 04
02 O que uma captura consegue ver 04
03 Preparação no Windows 04
04 Tempo e fuso: o mesmo pacote, dois relógios 06
05 Qualidade e preservação 06
06 Ler as camadas de um pacote 07
07 Filtros: captura versus exibição 08

PROTOCOLOS

08 DNS: da consulta à resposta 09
09 TCP: conexão e encerramento 10
10 Conexão recusada ou sem resposta 11
11 Retransmissão, janela e lentidão 11
12 Ferramentas de análise 13
13 HTTP, TLS e QUIC 14

WINDOWS E DOMÍNIO

14 Tráfego de Windows e domínio 16
15 Correlacionar pacote e evento 17

LABORATÓRIO

16 As capturas sintéticas 19
17 Laboratório: DNS e descoberta do DC 21
18 Laboratório: TCP e compartilhamento 21
19 Captura com prazo: dumpcap e buffer circular 22
20 Scripts de análise e validação 25

CASOS

21 Casos de diagnóstico resolvidos 28
22 Caso integrado com perguntas 29

APÊNDICES

A Formulários 31
B Referência de filtros validados 32
C Glossário 33
D Fontes oficiais 34
E Registro de edições e verificação 35

01 Método de investigação



Guarde a captura original. Filtrar para análise é diferente de exportar só alguns pacotes: uma exportação parcial pode perder o handshake, a resolução DNS ou as respostas necessárias à interpretação [1]. O capítulo 21 mostra o efeito na prática com a captura `10_meio_da_sessao`.

> **PRINCÍPIO** Escreva o que foi observado e onde. "Não observei resposta no ponto X durante Y segundos" é um fato; "o servidor não respondeu" só é fato se houver evidência do lado do servidor.

02 O que uma captura consegue ver

Uma interface observa o tráfego disponível naquele ponto. Numa rede com switch, capturar na estação não revela as conversas entre outros equipamentos. Modo promíscuo não faz o switch entregar tráfego que ele não encaminhou à porta [2].

Posição	Visibilidade e limite
Cliente	Solicitações e respostas vistas pelo cliente; VPN, virtualização e offloads podem mudar a aparência.
Servidor	Tráfego que entra e sai da interface capturada; pode excluir fluxos de outro nó do serviço.
SPAN / espelhamento	Cópia configurada de portas ou VLANs; pode perder pacotes em sobrecarga e duplicar quadros.
TAP	Observação do enlace correspondente; depende do equipamento e da coleta.
Wi-Fi	Modo gerenciado não equivale a modo monitor; exige suporte do driver, do adaptador e do canal.
Loopback	Tráfego da máquina para ela mesma; exige a interface de loopback, não a placa de rede.

Escolha do ponto

Para "o servidor respondeu?", capture no cliente e, se necessário, no servidor. Se o pacote aparece no servidor mas não no cliente, compare horários e caminho; a diferença reduz as hipóteses, mas ainda exige avaliar perda na própria captura. Em VPN, o adaptador físico pode mostrar só o túnel cifrado e o virtual expor o tráfego interno autorizado: identifique os dois antes de coletar.

03 Preparação no Windows

Obtenha o Wireshark pelos canais oficiais. No Windows, a captura normalmente usa o Npcap; as permissões dependem das opções de instalação e da política local. Uma interface sem pacotes não prova ausência de comunicação [1, 2].

01 Registre versão do Wireshark, sistema, interface, IP, VPN e horário com fuso.

02 **Confira o relógio da máquina que captura.** O carimbo de tempo vem do sistema operacional via Npcap; o Wireshark não cria horários [20]. Relógio errado invalida a correlação com logs (capítulo 4).

03 **Desligue a resolução de nomes de rede.** A documentação avisa que a resolução DNS pode acrescentar pacotes à captura e criar efeito observador [22]. Controle em *View → Name Resolution* ("Enable for Network Layer") e nas preferências [23]. Para TShark, use `-n`, como em todas as saídas desta apostila.

04 Selecione a interface pela atividade e pelo endereço esperado. Para localhost, use a interface de loopback.

05 Inicie antes da ação, reproduza uma vez, aguarde o resultado e pare.

06 Salve em pcapng com nome único: `caso_host_interface_dataUTC.pcapng`.

DADOS SENSÍVEIS Uma captura pode conter nomes, cookies, arquivos e credenciais quando não cifrados. Use conteúdo fictício nos testes, limite o acesso aos arquivos e não compartilhe PCAP de produção sem revisão.

04 Tempo e fuso: o mesmo pacote, dois relógios

O libpcap e o pcapng gravam o momento de chegada em UTC, e o Wireshark armazena internamente em UTC, mas **exibe sempre no horário local** do computador de quem abre o arquivo [21]. Duas pessoas em fusos diferentes veem horários diferentes para o mesmo quadro. Os logs do Windows, por sua vez, guardam o SystemTime em UTC.

EXECUTADO O mesmo arquivo lido com o fuso de São Paulo (capinfos) e em UTC (TShark, opção `-t ud`):

```
$ TZ=America/Sao_Paulo capinfos -a -e 05_lentidao_dns.pcapng
File name:           05_lentidao_dns.pcapng
First packet time:   2026-10-29 11:00:00.000000
Last packet time:    2026-10-29 11:00:02.051000

$ tshark -r 05_lentidao_dns.pcapng -t ud -c 2
    1 2026-10-29 14:00:00.000000  10.10.10.20 → 10.10.10.10  DNS 75 Standard query 0x2b01 A
app.ad.lab.test
    2 2026-10-29 14:00:02.000000  10.10.10.10 → 10.10.10.20  DNS 106 Standard query response 0x2b01 A
app.ad.lab.test A 10.10.10.40
```

O primeiro pacote aparece às 11:00 no horário de Brasília e às 14:00 em UTC. Nada mudou no arquivo; mudou só a exibição. Numa correlação com o evento 4624 de um servidor, comparar 11:00 com 14:00Z produziria uma diferença falsa de três horas.

Onde	Como ver em UTC
TShark	`-t ud` (data e hora UTC) ou o campo `frame.time_utc` com `-T fields`.
Wireshark	*View → Time Display Format*; nas versões atuais há opções em UTC. O rótulo exato varia: confira na sua versão.
Relatório	Registre sempre UTC e, se citar horário local, o fuso junto.

BOA PRÁTICA A documentação do Wireshark recomenda relógios sincronizados por NTP e registra que muitas organizações configuram servidores e equipamentos de rede em UTC justamente para simplificar a correlação [21].

05 Qualidade e preservação

Em *Statistics → Capture File Properties* (ou `capinfos` [28]), examine duração, interfaces e estatísticas. Nem todo formato ou driver registra perdas; ausência de contador não prova captura sem perda. Um snaplen reduzido trunca o conteúdo e impede a análise da aplicação [1, 18]. O dumpcap informa recebidos e descartados ao final (capítulo 19).

Artefatos comuns

- **Checksum "incorreto" em pacotes de saída** quando a captura precede o cálculo feito pela placa (offload). Confirme o offload e compare com outro ponto antes de diagnosticar corrupção [3].
- **Segmentação por hardware** pode produzir tamanhos diferentes dos que passam no fio.
- **Captura iniciada no meio da sessão** perde SYN, parâmetros TCP e o início da negociação TLS, e distorce a numeração relativa (capítulo 21, caso D).
- **Duplicação por espelhamento** pode parecer retransmissão.

```
Get-FileHash C:\LabCapturas\caso01.pcapng -Algorithm SHA256       # Windows
sha256sum caso01.pcapng                                            # Linux
```

O hash verifica que os bytes não mudaram depois da coleta; não comprova que a captura representa toda a rede. Preserve o original e analise uma cópia.

Dados a registrar

Host coletor, interface, ponto, filtro de captura, início e fim em UTC, versão da ferramenta, snaplen, perda reportada, caminho do arquivo e SHA256. Documente o que ficou de fora por filtro ou por posição. `coletar_captura.ps1` grava isso num manifesto (capítulo 19).

06 Ler as camadas de um pacote

Camada	Pergunta prática
Frame	Quando foi observado? Quantos bytes foram capturados? Em qual interface?
Ethernet	Quem enviou no enlace local? O MAC pode ser do gateway, não do servidor remoto.
ARP / ICMPv6	Houve descoberta do vizinho? ARP é IPv4; o IPv6 usa Neighbor Discovery.
IPv4 / IPv6	Origem, destino e fragmentação; o endereço pode ser traduzido por NAT em outro ponto.
TCP / UDP	Portas e conversa de transporte; porta sozinha não prova a aplicação (capítulo 7).
Aplicação	Que solicitação ou resposta foi decodificada? Está cifrada? O dissector reconheceu corretamente?

A lista de pacotes resume; os detalhes mostram os campos decodificados; os bytes permitem conferir. Expanda o campo e use *Apply as Filter* para obter o nome exato na versão instalada. Não interprete apenas a coluna Info [1].

07 Filtros: captura versus exibição

O filtro de captura decide o que entra no arquivo; o de exibição seleciona o que aparece na análise. As linguagens são diferentes, e um filtro de captura estreito descarta a evidência antes que você perceba [4, 5].

Captura (BPF)	Exibição (Wireshark)
`host 10.10.10.20`	`ip.addr == 10.10.10.20`
`tcp port 445`	`tcp.port == 445`
`port 53`	`udp.port == 53 || tcp.port == 53`
`host 10.10.10.20 and port 53`	`ip.addr == 10.10.10.20 && dns`
`net 10.10.10.0/24`	`ip.addr == 10.10.10.0/24`

EXECUTADO Os cinco filtros BPF compilaram no dumpcap 4.2.2; um filtro propositalmente errado (`prot 53`) foi rejeitado. Os filtros de exibição estão no Apêndice B.

Porta 53 não é DNS

A equivalência da tabela é de intenção, não de resultado. Na captura `01_dns`, os dois filtros que parecem iguais selecionam quantidades diferentes de quadros:

EXECUTADO Contagem por filtro em `01_dns.pcapng` (19 quadros):

```
Filtro de exibicao                    Quadros
dns                                   12
udp.port == 53 || tcp.port == 53      19
dns.flags.response == 0               6
dns.flags.response == 1               6
dns.flags.rcode != 0                  2
dns.flags.rcode == 3                  1
dns.flags.truncated == 1              1
tcp.port == 53 && !dns                7
```

Os sete quadros de `tcp.port == 53 && !dns` são o handshake, os ACKs e o encerramento da conversa TCP que carregou a consulta DNS: estão na porta 53, mas não são mensagens DNS. O filtro `dns` depende da decodificação; a porta é só uma pista.

Regras de escrita

- `ip.addr` é IPv4; use `ipv6.addr` para IPv6.
- Para excluir um host, use `!(ip.addr == 10.10.10.20)`: a negação explícita deixa claro que se exclui todo quadro com aquele endereço em qualquer campo.
- Use parênteses em expressões compostas.
- `tcp.stream` é um índice daquela captura, não um identificador global: `tcp.stream == 0` é a primeira conversa TCP de cada arquivo.
- Valide o filtro no campo de exibição (fica verde) antes de transformá-lo em procedimento.

08 DNS: da consulta à resposta

Investigue nome, tipo, servidor consultado, resposta, RCODE, TTL e tempo. Relacione consulta e resposta pelo transporte e pelo ID da transação; IDs podem se repetir em conversas diferentes. O Wireshark faz esse pareamento e expõe dois campos úteis: `dns.response_to` (quadro da consulta) e `dns.time` (tempo entre consulta e resposta) [6, 24].

Observação	Interpretação inicial
NXDOMAIN / RCODE 3	O nome não existe segundo aquela resposta [6].
SERVFAIL / RCODE 2	O servidor não conseguiu concluir; não equivale a nome inexistente.
NOERROR sem resposta do tipo	Pode indicar ausência daquele tipo; analise autoridade e contexto.
TC = 1 (truncada)	Acompanhe a nova consulta, normalmente por TCP, antes de declarar falha [7].
Nenhuma consulta visível	Cache, outro resolvedor, DNS cifrado, outra interface ou filtro podem explicar.

EXECUTADO **SINTÉTICO** Respostas de `01_dns.pcapng` lidas campo a campo (tipo 1 = A, 33 = SRV, 16 = TXT):

```
Respostas DNS (campos):
frame.number dns.response_to dns.qry.name dns.qry.type dns.flags.rcode dns.flags.truncated dns.time
dns.a dns.resp.ttl
2 1 fs01.ad.lab.test 1 0 False 0.002000000 10.10.10.30 1200
4 3 _ldap._tcp.dc._msdcs.ad.lab.test 33 0 False 0.003000000  600
6 5 fs02.ad.lab.test 1 3 False 0.002000000
8 7 app.parceiro.test 1 2 False 0.900000000
10 9 grande.ad.lab.test 16 0 True 0.002000000
15 14 grande.ad.lab.test 16 0 False 0.002000000  300

Conversa TCP/53 depois da resposta truncada:
   11   4.004000  10.10.10.20 → 10.10.10.10  TCP 54 53010 → 53 [SYN] Seq=0 Win=64240 Len=0
   12   4.005000  10.10.10.10 → 10.10.10.20  TCP 54 53 → 53010 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0
   13   4.006000  10.10.10.20 → 10.10.10.10  TCP 54 53010 → 53 [ACK] Seq=1 Ack=1 Win=64240 Len=0
   14   4.007000  10.10.10.20 → 10.10.10.10  DNS 92 Standard query 0x1a06 TXT grande.ad.lab.test
   15   4.009000  10.10.10.10 → 10.10.10.20  DNS 725 Standard query response 0x1a06 TXT
grande.ad.lab.test TXT
   16   4.010000  10.10.10.20 → 10.10.10.10  TCP 54 53010 → 53 [ACK] Seq=39 Ack=672 Win=64240 Len=0
   17   4.011000  10.10.10.20 → 10.10.10.10  TCP 54 53010 → 53 [FIN, ACK] Seq=39 Ack=672 Win=64240
Len=0
   18   4.012000  10.10.10.10 → 10.10.10.20  TCP 54 53 → 53010 [FIN, ACK] Seq=672 Ack=40 Win=64240
Len=0
   19   4.013000  10.10.10.20 → 10.10.10.10  TCP 54 53010 → 53 [ACK] Seq=40 Ack=673 Win=64240 Len=0
```

Como ler a saída

- Quadro 2: `fs01.ad.lab.test` resolveu para 10.10.10.30 em 2 ms, TTL 1200 s.
- Quadro 4: o registro SRV aponta `dc01.ad.lab.test` na porta 389: é assim que o cliente localiza o DC.
- Quadro 6: RCODE 3 (NXDOMAIN) para `fs02`, um nome inexistente.
- Quadro 8: RCODE 2 (SERVFAIL) depois de 900 ms; servidor que não concluiu, não nome inexistente.
- Quadro 10: resposta truncada (`True`), seguida de uma conversa TCP/53 completa em que a resposta chegou (quadro 15). Parar no quadro 10 levaria a uma conclusão errada.

DNS sobre HTTPS ou TLS oculta o conteúdo no transporte cifrado: falta de UDP/53 não significa ausência de resolução. O TTL observado não prova sozinho quando a zona foi alterada. Para separar cache de zona desatualizada, compare resolvedor e autoritativo com consultas específicas e registre a origem de cada resposta.

09 TCP: conexão e encerramento

No estabelecimento comum: o cliente envia SYN, o servidor responde SYN/ACK, o cliente confirma com ACK. SYN e FIN consomem espaço de sequência; o ACK confirma bytes recebidos pela pilha TCP, não o processamento pela aplicação [8].

EXECUTADO **SINTÉTICO** `02_tcp_ok.pcapng` e o iRTT calculado pelo Wireshark:

```
02_tcp_ok (handshake, SMB2 Negotiate e encerramento):
    1   0.000000  10.10.10.20 → 10.10.10.30  TCP 54 51000 → 445 [SYN] Seq=0 Win=64240 Len=0
    2   0.003000  10.10.10.30 → 10.10.10.20  TCP 54 445 → 51000 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0
    3   0.004000  10.10.10.20 → 10.10.10.30  TCP 54 51000 → 445 [ACK] Seq=1 Ack=1 Win=64240 Len=0
    4   0.006000  10.10.10.20 → 10.10.10.30  SMB2 162 Negotiate Protocol Request
    5   0.015000  10.10.10.30 → 10.10.10.20  SMB2 186 Negotiate Protocol Response
    6   0.016000  10.10.10.20 → 10.10.10.30  TCP 54 51000 → 445 [ACK] Seq=109 Ack=133 Win=64240 Len=0
    7   0.100000  10.10.10.20 → 10.10.10.30  TCP 54 51000 → 445 [FIN, ACK] Seq=109 Ack=133 Win=64240
Len=0
    8   0.101000  10.10.10.30 → 10.10.10.20  TCP 54 445 → 51000 [FIN, ACK] Seq=133 Ack=110 Win=64240
Len=0
    9   0.102000  10.10.10.20 → 10.10.10.30  TCP 54 51000 → 445 [ACK] Seq=110 Ack=134 Win=64240 Len=0

iRTT calculado pelo Wireshark:
3       0.004000000
```

Os quadros 1 a 3 são o handshake; o iRTT (`tcp.analysis.initial_rtt`) de 4 ms é o tempo entre o SYN e o ACK final, medido neste ponto. Os quadros 4 e 5 mostram a negociação SMB2. Os quadros 7 a 9 são o encerramento: cada FIN fecha uma direção, e o fechamento completo pode usar mais de três quadros.

> **LIMITE** O handshake indica que o transporte avançou. Não prova autenticação no domínio, autorização de leitura ou funcionamento da aplicação inteira.

10 Conexão recusada ou sem resposta

Observado	Hipóteses e próximo teste
SYN seguido de RST	Recusa ativa, compatível com porta fechada ou rejeição por dispositivo intermediário. Identifique a origem observada do RST e confira servidor e firewall.
SYN repetido sem resposta	Descarte, rota incorreta, host indisponível ou perda na captura. Não comprova firewall.
SYN/ACK sem ACK visível	Caminho de volta, cliente, filtro e perda na coleta.
Handshake completo e RST depois	Examine a fase da aplicação e os logs; não é "porta fechada".
ICMP unreachable	Leia tipo e código e o pacote citado; distinga a origem do ICMP do destino final.

EXECUTADO **SINTÉTICO** `03_tcp_rst` (recusa) e `04_syn_sem_resposta` (tentativas sem retorno):

```
03_tcp_rst:
    1   0.000000  10.10.10.20 → 10.10.10.30  TCP 54 51010 → 8080 [SYN] Seq=0 Win=64240 Len=0
    2   0.001000  10.10.10.30 → 10.10.10.20  TCP 54 8080 → 51010 [RST, ACK] Seq=1 Ack=1 Win=64240 Len=0

04_syn_sem_resposta:
    1   0.000000  10.10.10.20 → 10.10.10.30  TCP 54 51020 → 445 [SYN] Seq=0 Win=64240 Len=0
    2   1.000000  10.10.10.20 → 10.10.10.30  TCP 54 [TCP Retransmission] 51020 → 445 [SYN] Seq=0
Win=64240 Len=0
    3   3.000000  10.10.10.20 → 10.10.10.30  TCP 54 [TCP Retransmission] 51020 → 445 [SYN] Seq=0
Win=64240 Len=0
```

Em `04`, o Wireshark marca os SYN repetidos como *TCP Retransmission* e os intervalos dobram (1 s, depois 2 s). A conclusão correta é "três tentativas sem retorno observadas no cliente durante 3 s". A recusa real também aparece no capítulo 19, numa captura de loopback em que cada SYN para uma porta fechada recebe RST.

> **TESTES DE DIAGNÓSTICO** Um teste de conexão TCP abre e fecha a conexão sem conversa de aplicação. O `Test-NetConnection` do Windows, além do teste de porta, faz teste de ping, e a saída detalhada inclui resultados de resolução de nome [25]. Na captura, separe esses pacotes dos da operação real do usuário.

11 Retransmissão, janela e lentidão

Os campos `tcp.analysis.*` são inferências do Wireshark sobre a sequência observada. Podem ser afetados por início tardio, perda na captura, reordenação e duplicação do espelhamento. O Expert Information é ponto de partida, não diagnóstico [9, 10].

EXECUTADO **SINTÉTICO** `06_retransmissao.pcapng`, captura no cliente desde o SYN:

```
06_retransmissao (captura no cliente, desde o SYN):
    1   0.000000  10.10.10.20 → 10.10.10.30  TCP 54 51040 → 40000 [SYN] Seq=0 Win=64240 Len=0
    2   0.002000  10.10.10.30 → 10.10.10.20  TCP 54 40000 → 51040 [SYN, ACK] Seq=0 Ack=1 Win=64240
Len=0
    3   0.003000  10.10.10.20 → 10.10.10.30  TCP 54 51040 → 40000 [ACK] Seq=1 Ack=1 Win=64240 Len=0
    4   0.010000  10.10.10.20 → 10.10.10.30  TCP 1054 51040 → 40000 [PSH, ACK] Seq=1 Ack=1 Win=64240
Len=1000 [TCP segment of a reassembled PDU]
    5   0.011000  10.10.10.20 → 10.10.10.30  TCP 1054 51040 → 40000 [PSH, ACK] Seq=1001 Ack=1 Win=64240
Len=1000 [TCP segment of a reassembled PDU]
    6   0.012000  10.10.10.20 → 10.10.10.30  TCP 1054 51040 → 40000 [PSH, ACK] Seq=2001 Ack=1 Win=64240
Len=1000 [TCP segment of a reassembled PDU]
    7   0.013000  10.10.10.20 → 10.10.10.30  TCP 1054 51040 → 40000 [PSH, ACK] Seq=3001 Ack=1 Win=64240
Len=1000 [TCP segment of a reassembled PDU]
    8   0.016000  10.10.10.30 → 10.10.10.20  TCP 54 40000 → 51040 [ACK] Seq=1 Ack=1001 Win=64240 Len=0
    9   0.018000  10.10.10.30 → 10.10.10.20  TCP 54 [TCP Dup ACK 8#1] 40000 → 51040 [ACK] Seq=1
Ack=1001 Win=64240 Len=0
   10   0.020000  10.10.10.30 → 10.10.10.20  TCP 54 [TCP Dup ACK 8#2] 40000 → 51040 [ACK] Seq=1
Ack=1001 Win=64240 Len=0
   11   0.022000  10.10.10.30 → 10.10.10.20  TCP 54 [TCP Dup ACK 8#3] 40000 → 51040 [ACK] Seq=1
Ack=1001 Win=64240 Len=0
   12   0.024000  10.10.10.20 → 10.10.10.30  TCP 1054 [TCP Fast Retransmission] 51040 → 40000 [PSH,
ACK] Seq=1001 Ack=1 Win=64240 Len=1000 [TCP segment of a reassembled PDU]
   13   0.026000  10.10.10.30 → 10.10.10.20  TCP 54 40000 → 51040 [ACK] Seq=1 Ack=4001 Win=64240 Len=0
   14   0.126000  10.10.10.20 → 10.10.10.30  TCP 1054 51040 → 40000 [PSH, ACK] Seq=4001 Ack=1 Win=64240
Len=1000 [TCP segment of a reassembled PDU]
   15   0.127000  10.10.10.30 → 10.10.10.20  TCP 54 [TCP ZeroWindow] 40000 → 51040 [ACK] Seq=1 Ack=5001
Win=0 Len=0
   16   0.626000  10.10.10.20 → 10.10.10.30  TCP 55 [TCP ZeroWindowProbe] 51040 → 40000 [PSH, ACK]
Seq=5001 Ack=1 Win=64240 Len=1 [TCP segment of a reassembled PDU]
   17   0.627000  10.10.10.30 → 10.10.10.20  TCP 54 [TCP ZeroWindowProbeAck] [TCP ZeroWindow] 40000 →
51040 [ACK] Seq=1 Ack=5001 Win=0 Len=0
   18   1.326000  10.10.10.30 → 10.10.10.20  TCP 54 [TCP Window Update] 40000 → 51040 [ACK] Seq=1
Ack=5001 Win=64240 Len=0

Filtro                              Quadros
tcp.analysis.retransmission         1
tcp.analysis.fast_retransmission    1
tcp.analysis.duplicate_ack          3
tcp.analysis.zero_window            2
tcp.analysis.zero_window_probe      1
tcp.analysis.window_update          1
```

Como ler a saída

- Quadros 8 a 11: o servidor confirma até o byte 1001 e repete esse ACK três vezes (*Dup ACK*): falta o segmento que começa em 1001.
- Quadro 12: o cliente reenvia esse segmento (*Fast Retransmission*). Como a captura é no cliente, o quadro 5 aparece mesmo tendo se perdido depois: o ponto de captura vê o envio, não a chegada.
- Quadros 15 a 18: o servidor anuncia janela zero, o cliente sonda com 1 byte (*ZeroWindowProbe*) e só 1,2 s depois chega o *Window Update*. Janela zero indica receptor sem espaço para receber naquele sentido; não significa falta de largura de banda.

Separar atrasos

Marque consulta DNS, resposta DNS, SYN, SYN/ACK, solicitação e início da resposta. O tempo de pedido e resposta observado inclui rede, filas e dependências, não só processamento do servidor; o RTT do TCP não é tempo de execução da aplicação. Compare uma operação normal e uma lenta, no mesmo ponto e com várias amostras: uma retransmissão isolada não demonstra a causa de uma reclamação. O caso A (capítulo 21) aplica isso com as medidas do próprio Wireshark.

12 Ferramentas de análise

Recurso	TShark equivalente	Uso prático
Protocol Hierarchy	`-q -z io,phs`	Visão dos protocolos decodificados; tráfego cifrado limita o detalhe.
Conversations / Endpoints	`-q -z conv,tcp` · `-z endpoints,ip`	Pares, volume e duração; confira se as estatísticas respeitam o filtro.
Expert Information	`-q -z expert,note`	Localizar sinais e verificar o contexto manualmente.
I/O Graphs	`-q -z io,stat,1`	Comparar volume e eventos ao longo do tempo com intervalo consistente.
Follow TCP/UDP Stream	`-q -z follow,tcp,ascii,0`	Reunir os dados de uma conversa; não decifra TLS.
Flow Graph	—	Visualizar ordem e direção dos pacotes selecionados.

EXECUTADO **SINTÉTICO** Hierarquia e conversas de `07_smb_acesso_negado.pcapng`; e o Expert Information de `06_retransmissao`:

```
Hierarquia de protocolos:
Protocol Hierarchy Statistics
Filter:
eth                                      frames:14 bytes:1678
  ip                                     frames:14 bytes:1678
    udp                                  frames:2 bytes:184
      dns                                frames:2 bytes:184
    tcp                                  frames:12 bytes:1494
      nbss                               frames:8 bytes:1278
        smb2                             frames:8 bytes:1278

Conversas TCP:
TCP Conversations
Filter:<No Filter>
                                                          |       <-      | |       ->      | |
Total     |    Relative    |   Duration   |
                                                          | Frames  Bytes | | Frames  Bytes | |
Frames  Bytes |      Start     |              |
10.10.10.20:51050          <-> 10.10.10.30:445                  5 638 bytes       7 856 bytes      12
1494 bytes     0.010000000         0.0510

Expert Information (resumo):
Warns (2)
=============
   Frequency      Group           Protocol  Summary
           2   Sequence                TCP  TCP Zero Window segment
Notes (7)
=============
   Frequency      Group           Protocol  Summary
           1   Sequence                TCP  Duplicate ACK (#1)
           1   Sequence                TCP  Duplicate ACK (#2)
           1   Sequence                TCP  Duplicate ACK (#3)
           1   Sequence                TCP  This frame is a (suspected) fast retransmission
           1   Sequence                TCP  This frame is a (suspected) retransmission
           1   Sequence                TCP  TCP Zero Window Probe
           1   Sequence                TCP  ACK to a TCP Zero Window Probe
```

Ao usar Follow Stream, confira a conversa e os sentidos selecionados. Uma conversa pode carregar várias requisições; o HTTP/2 multiplexa operações e não deve ser lido como troca linear [11].

Rotina em cinco minutos

Propriedades da captura → endpoints e conversas → conversa do sintoma → remover filtros que escondem o transporte → sequência e intervalos → anotar os quadros de evidência. Depois aprofunde a hipótese; evite começar procurando só os quadros coloridos de vermelho.

13 HTTP, TLS e QUIC

HTTP em claro expõe método, caminho, status e conteúdo. O TLS protege os dados da aplicação; sem os segredos da sessão, a captura normalmente não permite ler esse conteúdo [12, 13].

EXECUTADO SINTÉTICO `09_tls13.pcapng`: o que ainda é visível num handshake TLS 1.3:

```
09_tls13:
    1   0.000000  10.10.10.20 → 10.10.10.40  TCP 54 51060 → 443 [SYN] Seq=0 Win=64240 Len=0
    2   0.002000  10.10.10.40 → 10.10.10.20  TCP 54 443 → 51060 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0
    3   0.003000  10.10.10.20 → 10.10.10.40  TCP 54 51060 → 443 [ACK] Seq=1 Ack=1 Win=64240 Len=0
    4   0.004000  10.10.10.20 → 10.10.10.40  TLSv1 139 Client Hello (SNI=app.ad.lab.test)
    5   0.010000  10.10.10.40 → 10.10.10.20  TLSv1.3 109 Server Hello
    6   0.011000  10.10.10.40 → 10.10.10.20  TLSv1.3 123 Application Data
    7   0.013000  10.10.10.20 → 10.10.10.40  TLSv1.3 123 Application Data
    8   0.020000  10.10.10.40 → 10.10.10.20  TLSv1.3 123 Application Data
    9   0.021000  10.10.10.20 → 10.10.10.40  TCP 54 51060 → 443 [ACK] Seq=155 Ack=194 Win=64240 Len=0

frame.number tls.record.version tls.handshake.type tls.handshake.extensions_server_name
tls.handshake.extensions.supported_version
4 0x0301 1 app.ad.lab.test 0x0304
5 0x0303 2  0x0304
Quadros com certificado visivel (tls.handshake.type == 11): 0
```

Como ler a saída CORREÇÃO

- ClientHello e ServerHello vão em claro: dá para ver o SNI (`app.ad.lab.test`) e a versão negociada pela extensão `supported_versions` (0x0304 = TLS 1.3).
- O ClientHello aparece como "TLSv1" porque o campo de versão da camada de registro usa um valor legado por compatibilidade; a versão real está na extensão [13].
- **No TLS 1.3, o certificado do servidor vai sempre cifrado**: todas as mensagens de handshake depois do ServerHello são protegidas [13]. A contagem de `tls.handshake.type == 11` é zero. A Edição 1 dizia que o certificado "não fica necessariamente visível"; em TLS 1.3 ele nunca fica.
- O SNI pode faltar, por exemplo com ECH. UDP/443 é uma pista, não prova isolada de QUIC.

Decifrar em laboratório

Aplicações compatíveis podem gerar um arquivo de segredos de sessão para análise autorizada. Configure-o em *Preferences → Protocols → TLS → (Pre)-Master-Secret log filename* e reabra a captura correspondente. Possuir a chave privada do servidor não basta para sessões com sigilo futuro [12]. Esses segredos dão acesso ao conteúdo das sessões: use só tráfego fictício, acesso restrito e retenção definida. Nesta edição não houve descriptografia.

```
http.request || http.response
tls.handshake.type == 1
tls.handshake.extensions_server_name == "app.ad.lab.test"
tls || quic
```

14 Tráfego de Windows e domínio

Serviço e porta usual	Pergunta investigativa
DNS: TCP/UDP 53	O cliente localizou os registros do domínio (SRV, A)?
Kerberos: TCP/UDP 88	Houve solicitação e resposta do KDC? Com erro?
Troca de senha Kerberos: TCP/UDP 464	A alteração de senha chegou ao DC? NOVO
LDAP: TCP 389; UDP 389 (CLDAP)	O cliente alcança o serviço? A descoberta do DC por CLDAP respondeu? A troca está assinada ou selada?
LDAPS: TCP 636	A negociação TLS foi observada?
Catálogo global: TCP 3268 / 3269 (SSL)	Consultas que atravessam domínios da floresta NOVO
SMB: TCP 445	Sessão, autenticação e operação avançam até qual fase?
RPC: TCP 135 e faixa dinâmica	Há endpoint mapper e conectividade com a porta negociada? Desde o Windows Server 2008 e o Vista, a faixa dinâmica padrão é 49152–65535 [15].
Tempo: UDP 123	A fonte de tempo e o estado dos relógios estão coerentes?

Portas do artigo de serviços e requisitos de porta da Microsoft [14, 15]. A própria Microsoft avisa que a lista serve para entender dependências e não deve ser usada diretamente como regra de firewall.

Filtros que respondem às perguntas

Filtro	O que seleciona
`dns || kerberos || ldap || smb2 || dcerpc`	Aplicações de domínio decodificadas. Esconde SYN, ACK e retransmissões: volte ao transporte quando necessário.
`kerberos.error_code == 24`	KRB-ERROR KDC_ERR_PREAUTH_FAILED: pré-autenticação inválida, em geral senha errada. É o 0x18 do evento 4771 [27].
`cldap`	Descoberta do DC por LDAP sobre UDP.
`ntlmssp`	Autenticação NTLM dentro de SMB, HTTP ou outros: indica que o Kerberos não foi usado naquela troca.
`smb2.nt_status != 0`	Respostas SMB2 com qualquer status diferente de sucesso.
`smb2.nt_status == 0xc0000022`	STATUS_ACCESS_DENIED [26].

EXECUTADO SINTÉTICO `07_smb_acesso_negado` e `08_kerberos_preauth`:

```
07_smb_acesso_negado:
    1   0.000000  10.10.10.20 → 10.10.10.10  DNS 76 Standard query 0x3c01 A fs01.ad.lab.test
    2   0.002000  10.10.10.10 → 10.10.10.20  DNS 108 Standard query response 0x3c01 A fs01.ad.lab.test
A 10.10.10.30
    3   0.010000  10.10.10.20 → 10.10.10.30  TCP 54 51050 → 445 [SYN] Seq=0 Win=64240 Len=0
    4   0.012000  10.10.10.30 → 10.10.10.20  TCP 54 445 → 51050 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0
    5   0.013000  10.10.10.20 → 10.10.10.30  TCP 54 51050 → 445 [ACK] Seq=1 Ack=1 Win=64240 Len=0
    6   0.015000  10.10.10.20 → 10.10.10.30  SMB2 162 Negotiate Protocol Request
    7   0.020000  10.10.10.30 → 10.10.10.20  SMB2 186 Negotiate Protocol Response
    8   0.025000  10.10.10.20 → 10.10.10.30  SMB2 146 Session Setup Request
    9   0.040000  10.10.10.30 → 10.10.10.20  SMB2 130 Session Setup Response
   10   0.045000  10.10.10.20 → 10.10.10.30  SMB2 184 Tree Connect Request Tree: \
\fs01.ad.lab.test\LabShare
   11   0.050000  10.10.10.30 → 10.10.10.20  SMB2 138 Tree Connect Response
   12   0.055000  10.10.10.20 → 10.10.10.30  SMB2 202 Create Request File: restrito.txt
   13   0.060000  10.10.10.30 → 10.10.10.20  SMB2 130 Create Response, Error: STATUS_ACCESS_DENIED
   14   0.061000  10.10.10.20 → 10.10.10.30  TCP 54 51050 → 445 [ACK] Seq=479 Ack=369 Win=64240 Len=0

Respostas SMB2 com erro:
13 5 0xc0000022 3
Filtro smb2.nt_status == 0xc0000022: 1 quadro(s)

08_kerberos_preauth:
    1   0.000000  10.10.10.20 → 10.10.10.10  KRB5 210 AS-REQ
    2   0.004000  10.10.10.10 → 10.10.10.20  KRB5 181 KRB Error: KRB5KDC_ERR_PREAUTH_FAILED

frame.number kerberos.error_code kerberos.CNameString kerberos.realm
2 24 ana.lab AD.LAB.TEST
Filtro kerberos.error_code == 24: 1 quadro(s)
```

Em `07`, resolução, transporte, negociação, sessão e conexão ao compartilhamento avançam; a falha está na abertura do arquivo (Create), com STATUS_ACCESS_DENIED. Em `08`, o KDC recusa a pré-autenticação de `ana.lab`. Nenhum dos dois é problema de rede.

> **SMB CIFRADO** Assinatura SMB não é o mesmo que criptografia SMB [17]. Com criptografia, nomes de arquivo, comandos e status ficam dentro da camada cifrada e esses filtros não encontram nada; a análise volta a ser de transporte e tempo, completada pelos logs.

15 Correlacionar pacote e evento

Use três fontes: cliente, DC e servidor de destino. Sincronize horários e converta tudo para UTC (capítulo 4). O momento do pacote, o timestamp do evento e o recebimento pelo coletor não são a mesma coisa.

Rede	Log correspondente a investigar
AS-REQ / AS-REP (TGT)	4768 no DC, quando auditado.
KRB-ERROR 24 (pré-autenticação)	4771 com código 0x18 no DC [27].
TGS-REQ / TGS-REP (ticket de serviço)	4769 no DC; sucesso não prova acesso a arquivo [16].
NTLMSSP na sessão SMB	4776 na autoridade da conta (o DC, para conta de domínio).
Sessão SMB no destino	4624 tipo 3 no servidor.
Create com ACCESS_DENIED	5145 de falha no servidor, se a negação foi na camada do compartilhamento e Detailed File Share está auditado; negação NTFS exige a auditoria do objeto.

Não existe equivalência obrigatória de um pacote para um evento. Tickets e sessões são reutilizados, então uma leitura pode não gerar novo AS-REQ nem nova sessão. A apostila de investigação de logs no Windows e Active Directory detalha os eventos desta tabela.

> **CONCLUSÃO LIMITADA** "Foi observado TCP estabelecido para FS01 e tráfego SMB cifrado. O evento 4624 confirma uma sessão da conta no destino. A captura não permite identificar o arquivo acessado; a comprovação exige a auditoria do recurso."

16 As capturas sintéticas

As dez capturas do anexo foram montadas por `gerar_capturas.py` com a biblioteca Scapy [29]. Cada pacote é construído pelo script, com endereços fictícios (10.10.10.0/24, domínio `ad.lab.test`) e tempos escolhidos para cada caso, a partir de 29/10/2026 14:00:00 UTC. São arquivos pcapng legítimos e o Wireshark os decodifica normalmente, mas **não são tráfego de rede real**: servem para praticar leitura e filtros com resultado conhecido.

Arquivo	Quadros	Conteúdo
01_dns	19	A, SRV, NXDOMAIN, SERVFAIL e resposta truncada com nova consulta por TCP
02_tcp_ok	9	Handshake, SMB2 Negotiate e encerramento com FIN
03_tcp_rst	2	SYN recusado com RST
04_syn_sem_resposta	3	Três SYN sem retorno
05_lentidao_dns	8	DNS de 2 s antes de um HTTP rápido (caso A)
06_retransmissao	18	Dup ACK, retransmissão rápida, janela zero e sonda
07_smb_acesso_negado	14	DNS, TCP, SMB2 até Create com STATUS_ACCESS_DENIED
08_kerberos_preauth	2	AS-REQ e KRB-ERROR 24
09_tls13	9	ClientHello com SNI, ServerHello TLS 1.3 e dados cifrados
10_meio_da_sessao	13	A conversa de 06 sem os cinco primeiros quadros (caso D)

Reprodutibilidade e integridade

Campos aleatórios (GUID do SMB2, valores aleatórios do TLS) vêm de um gerador com semente fixa, e os horários internos dos protocolos são fixos. Rodar o script duas vezes produz arquivos byte a byte iguais, conferido nesta edição. Nenhuma captura tem quadro malformado ou erro de dissector no TShark 4.2.2.

EXECUTADO Versão e SHA256 (os mesmos de `SHA256SUMS.txt` no anexo):

```
TShark: TShark (Wireshark) 4.2.2 (Git v4.2.2 packaged as 4.2.2-1.1build3).

SHA256 das capturas
dcf1913d8252118d15ed5788094652a6e6967e020401d3e74dafc4b067224cd7  01_dns.pcapng
dceea618c1da0e1fc7b220d2cb993e17b7df6f560c41f440876c9b47d834b14c  02_tcp_ok.pcapng
e0bb687b69a7cb09087ee32d3cabafd46a0208d07c5830d6db4255a5df29a16e  03_tcp_rst.pcapng
b0e1c9e5f57a37c853146487983f90e44aec57ff14d122b3c8ee7200e7148b88  04_syn_sem_resposta.pcapng
002b039517103944e5e76c8df8ca5df72df66ff4115508e1933f2a8ff80f2a03  05_lentidao_dns.pcapng
1ac40a4bdffb163646b5a90daec7e2f161fbe44c81f5f62e97cc3480df896b46  06_retransmissao.pcapng
f5bd1597524341e98822d905d6d7b0678112bf020abd06c3ea5d62e47b912fbf  07_smb_acesso_negado.pcapng
716d537d147f0770db064ccf191753009ce082da7256f1d832f2400abf9851ef  08_kerberos_preauth.pcapng
5214a9aaacb39563444c32e822d145ce33e143b8ed52c0d4826b4a7de150491a  09_tls13.pcapng
a55844bc66493d51d44d5aa0e05c1ee2c328bf6135daf58af062f6884fbecf6f  10_meio_da_sessao.pcapng
```

Trecho de `gerar_capturas.py`: a captura do caso integrado.

```
def c07_smb_negado(pasta):
    p = [stamp(dns_q('cl01', 'dc01', 0x3c01, 'fs01.ad.lab.test'), 0.000),
         stamp(dns_r('cl01', 'dc01', 0x3c01, 'fs01.ad.lab.test',
                     an=DNSRR(rrname='fs01.ad.lab.test', type='A', ttl=1200, rdata=IPS['fs01'])),
0.002)]
    c = Conversa('cl01', 'fs01', 51050, 445)
    sid = 0x0000100000000041
```

```
    def req(cmd, mid, corpo, tid=0):
        return NBTSession() / S.SMB2_Header(Command=cmd, MID=mid, SessionId=sid if mid > 1 else 0,
TID=tid) / corpo
    def resp(cmd, mid, corpo, status=0, tid=0):
        return NBTSession() / S.SMB2_Header(Command=cmd, MID=mid, SessionId=sid if mid > 0 else 0,
TID=tid,
                                            Flags='SMB2_FLAGS_SERVER_TO_REDIR', Status=status) / corpo
    share = '\\\\fs01.ad.lab.test\\LabShare'
    seq = [(0.010, c.syn()), (0.012, c.synack()), (0.013, c.ack('c')),
           (0.015, c.dados('c', req(0x0000, 0, S.SMB2_Negotiate_Protocol_Request(Dialects=[0x0302,
0x0311])))),
           (0.020, c.dados('s', resp(0x0000, 0,
S.SMB2_Negotiate_Protocol_Response(DialectRevision=0x0311, ServerTime=FILETIME, ServerStartTime=0)))),
           (0.025, c.dados('c', req(0x0001, 1, S.SMB2_Session_Setup_Request()))),
           (0.040, c.dados('s', resp(0x0001, 1, S.SMB2_Session_Setup_Response()))),
           (0.045, c.dados('c', req(0x0003, 2, S.SMB2_Tree_Connect_Request(Buffer=[('Path',
share)])))),
           (0.050, c.dados('s', resp(0x0003, 2, S.SMB2_Tree_Connect_Response(ShareType='DISK'),
tid=1))),
           (0.055, c.dados('c', req(0x0005, 3, S.SMB2_Create_Request(Buffer=[('Name',
'restrito.txt')]), tid=1))),
           (0.060, c.dados('s', resp(0x0005, 3, S.SMB2_Error_Response(), status=0xC0000022, tid=1))),
           (0.061, c.ack('c'))]
    p += [stamp(pk, t) for t, pk in seq]
    salvar(pasta, '07_smb_acesso_negado.pcapng', p)
```

17 Laboratório: DNS e descoberta do DC

NÃO EXECUTADO Pré-requisitos: CL01 = 10.10.10.20, DC01/DNS = 10.10.10.10 e domínio `ad.lab.test` em rede isolada. Ajuste os endereços. Não é necessário modificar caches na primeira prática.

01 No Wireshark de CL01, desligue a resolução de nomes de rede e escolha exibição em UTC (capítulos 3 e 4).

02 Inicie a captura na interface de CL01 antes da consulta e anote o horário UTC.

03 Execute as consultas abaixo. Elas geram tráfego de diagnóstico.

04 Pare e salve. Filtre `dns` e depois o nome consultado; registre quadro de consulta, resposta, tipo, RCODE, `dns.time` e endereço retornado.

05 Confira se houve UDP, TCP ou outra tentativa. Sem tráfego, verifique interface, cache e comportamento da ferramenta antes de concluir falha.

```
Resolve-DnsName -Name fs01.ad.lab.test -Type A -Server 10.10.10.10 -DnsOnly
Resolve-DnsName -Name _ldap._tcp.dc._msdcs.ad.lab.test -Type SRV -Server 10.10.10.10 -DnsOnly
```

Registre A versus SRV: o primeiro busca endereço, o segundo descreve onde está um serviço. Uma resposta correta não comprova a conexão posterior ao serviço.

Antes do domínio: pratique com o anexo

Sem laboratório pronto, abra `01_dns.pcapng` e reproduza a tabela do capítulo 8 só com filtros: `dns.flags.rcode != 0`, `dns.flags.truncated == 1`, `dns.time > 0.5`. Depois compare com a saída do capítulo 8.

Entrega: pcapng, SHA256, comandos, tabela de consulta e resposta e explicação de eventual ausência.

18 Laboratório: TCP e compartilhamento

NÃO EXECUTADO Use FS01 com compartilhamento de teste e arquivo sem dados reais; a conta deve ter permissão de leitura. Auditoria do recurso é mudança de laboratório, opcional para observar o handshake.

```
Test-NetConnection fs01.ad.lab.test -Port 445
# Depois, numa ação separada e anotada:
Get-Content \\fs01.ad.lab.test\LabShare\leitura.txt
```

01 Capture as duas ações em intervalos anotados. O teste de porta não é a leitura do arquivo, e ele também gera ping e consulta de nome (capítulo 10).

02 Identifique a conversa por IP e portas e use *Follow* ou `tcp.stream`.

03 Localize handshake (se houver), negociação SMB, autenticação (`kerberos` ou `ntlmssp`) e operações quando decodificáveis.

04 Compare o resultado do comando com os eventos do servidor. Se já havia sessão SMB aberta, pode não haver novo handshake: registre esse contexto.

05 Repita uma vez com conta de teste sem acesso, em sessão separada, e procure `smb2.nt_status != 0`. Identifique em que etapa ocorre a negativa, sem confundi-la com falha de transporte.

Não remova criptografia nem políticas de segurança só para tornar o tráfego visível. Uma captura cifrada ainda sustenta análise de conectividade e tempo, completada pelos logs.

Entrega: dois fluxos identificados, resultado da aplicação, eventos relevantes e limites. Para treinar antes, use `07_smb_acesso_negado.pcapng`.

19 Captura com prazo: dumpcap e buffer circular

O dumpcap captura; o TShark lê e analisa [18, 19]. Uma captura sem prazo enche o disco e coleta mais dados sensíveis do que o necessário. Duas opções resolvem: `-a` encerra (por duração ou tamanho) e `-b` roda arquivos num buffer circular, mantendo só os N mais recentes [18].

Opção	Efeito [18]
`-D`	Lista as interfaces e sai.
`-i <interface>`	Interface (nome ou índice de `-D`).
`-f "<BPF>"`	Filtro de captura, entre aspas como um único argumento.
`-a duration:<s>`	Para depois de s segundos.
`-b filesize:<kB>` / `-b duration:<s>`	Troca de arquivo por tamanho ou por tempo.
`-b files:<n>`	Mantém só os n arquivos mais recentes; exige filesize, duration ou interval.
`-s <snaplen>`	Bytes por pacote; 0 (padrão) captura o pacote inteiro.

EXECUTADO **Captura real** na interface de loopback de Linux, com arquivos de 1 s, no máximo 3 arquivos e prazo de 5 s, enquanto um cliente tentava conectar repetidamente a uma porta fechada (127.0.0.1:9):

```
$ dumpcap -i lo -b duration:1 -b files:3 -a duration:5 -w anel/lab.pcapng
Capturing on 'Loopback: lo'
File: anel/lab_00001_20261029143311.pcapng
File: anel/lab_00002_20261029143313.pcapng
File: anel/lab_00003_20261029143314.pcapng
File: anel/lab_00004_20261029143315.pcapng
File: anel/lab_00005_20261029143316.pcapng
Packets captured: 49
Packets received/dropped on interface 'Loopback: lo': 49/0 (pcap:0/dumpcap:0/flushed:0/ps_ifdrop:0)
(100.0%)

$ ls anel/
lab_00003_20261029143314.pcapng
lab_00004_20261029143315.pcapng
lab_00005_20261029143316.pcapng

$ tshark -r anel/lab_00003_*.pcapng -c 6
    1 0.000000000    127.0.0.1 → 127.0.0.1    TCP 54 9 → 53990 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
    2 0.214215033    127.0.0.1 → 127.0.0.1    TCP 74 53998 → 9 [SYN] Seq=0 Win=65495 Len=0 MSS=65495
SACK_PERM TSval=1347598844 TSecr=0 WS=1024
    3 0.214222621    127.0.0.1 → 127.0.0.1    TCP 54 9 → 53998 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
    4 0.426486873    127.0.0.1 → 127.0.0.1    TCP 74 54006 → 9 [SYN] Seq=0 Win=65495 Len=0 MSS=65495
SACK_PERM TSval=411542866 TSecr=0 WS=1024
    5 0.426499610    127.0.0.1 → 127.0.0.1    TCP 54 9 → 54006 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
    6 0.640783484    127.0.0.1 → 127.0.0.1    TCP 74 54016 → 9 [SYN] Seq=0 Win=65495 Len=0 MSS=65495
SACK_PERM TSval=3814338219 TSecr=0 WS=1024
```

Duas lições desta saída

- **O buffer circular apaga.** Foram criados cinco arquivos, mas só restaram os três últimos: `-b files:3` descartou os dois primeiros. Use isso de propósito, para limitar disco, e saiba que o início do incidente pode ter sido apagado.

- **O corte entre arquivos parte conversas.** O primeiro quadro do arquivo 3 é um RST cujo SYN ficou no arquivo anterior, já apagado. Para analisar, junte os arquivos restantes em ordem (`mergecap`) e lembre que o início pode faltar.

Esta é também uma recusa real (capítulo 10): cada SYN para a porta 9 recebe RST/ACK imediato da própria pilha, com janela zero e sem dados.

No Windows

```
& "$env:ProgramFiles\Wireshark\dumpcap.exe" -D
& "$env:ProgramFiles\Wireshark\dumpcap.exe" -i 1 -a duration:120 -b filesize:100000 -b files:10 -w C:
\LabCapturas\caso.pcapng
$primeiro = (Get-ChildItem C:\LabCapturas\caso_*.pcapng | Sort-Object Name | Select-Object -First
1).FullName
& "$env:ProgramFiles\Wireshark\tshark.exe" -n -r $primeiro -Y "tcp.flags.reset == 1"
```

NÃO EXECUTADO Troque 1 pelo índice confirmado em `-D`. O TShark não expande curingas no `-r`: o arquivo é escolhido antes, pelo PowerShell. `coletar_captura.ps1` faz o mesmo e grava hash e manifesto.

POWERSHELL **NÃO EXECUTADO** coletar_captura.ps1

```
<#
.SYNOPSIS
  Captura com dumpcap em arquivos rotativos, com prazo, hash e manifesto (Windows).
.DESCRIPTION
  Grava apenas na pasta nova criada para o caso. Usa buffer circular (-b) e prazo (-a)
  para nunca deixar captura sem limite. Requer Wireshark/Npcap instalados e permissao de captura.
  ESTADO NESTA EDICAO: NAO EXECUTADO no Windows (apenas validacao de sintaxe).
  A mesma linha de dumpcap foi executada em Linux na interface de loopback (capitulo 20).
.EXAMPLE
  .\coletar_captura.ps1 -ListarInterfaces
  .\coletar_captura.ps1 -Caso LAB-01 -Interface 1 -Segundos 120 -Destino C:\LabCapturas
#>
[CmdletBinding(DefaultParameterSetName = 'Capturar')]
param(
    [Parameter(ParameterSetName = 'Listar')]
    [switch]$ListarInterfaces,

    [Parameter(Mandatory = $true, ParameterSetName = 'Capturar')]
    [ValidatePattern('^[A-Za-z0-9_-]+$')]
    [string]$Caso,

    [Parameter(Mandatory = $true, ParameterSetName = 'Capturar')]
    [string]$Interface,

    [Parameter(ParameterSetName = 'Capturar')]
    [ValidateRange(5, 86400)]
    [int]$Segundos = 120,

    [Parameter(ParameterSetName = 'Capturar')]
    [int]$ArquivoKB = 100000,

    [Parameter(ParameterSetName = 'Capturar')]
    [int]$Arquivos = 10,

    [Parameter(ParameterSetName = 'Capturar')]
    [string]$FiltroCaptura,

    [Parameter(ParameterSetName = 'Capturar')]
    [string]$Destino = 'C:\LabCapturas',

    [string]$Dumpcap = (Join-Path -Path $env:ProgramFiles -ChildPath 'Wireshark\dumpcap.exe')
```

```
)

if (-not (Test-Path -LiteralPath $Dumpcap)) {
    Write-Error "dumpcap nao encontrado em $Dumpcap. Informe -Dumpcap."
    exit 3
}

if ($ListarInterfaces) {
    & $Dumpcap -D
    exit $LASTEXITCODE
}

$inicio = (Get-Date).ToUniversalTime()
$pasta = Join-Path -Path $Destino -ChildPath ('{0}_{1}_{2}' -f $Caso, $env:COMPUTERNAME,
$inicio.ToString("yyyyMMdd'T'HHmmss'Z'"))
if (Test-Path -LiteralPath $pasta) { Write-Error "Pasta ja existe: $pasta"; exit 3 }
New-Item -ItemType Directory -Path $pasta | Out-Null

$base = Join-Path -Path $pasta -ChildPath ('{0}.pcapng' -f $Caso)
$argumentos = @('-i', $Interface, '-a', ('duration:' + $Segundos), '-b', ('filesize:' + $ArquivoKB), '-
b', ('files:' + $Arquivos), '-w', $base)
if ($FiltroCaptura) { $argumentos += @('-f', $FiltroCaptura) }

Write-Output ("Capturando por {0} s na interface {1}. Reproduza a acao agora." -f $Segundos,
$Interface)
$saida = & $Dumpcap @argumentos 2>&1
$codigo = $LASTEXITCODE
$fim = (Get-Date).ToUniversalTime()

$arquivos = @(Get-ChildItem -LiteralPath $pasta -Filter '*.pcapng' | Sort-Object Name | ForEach-Object
{
    [pscustomobject]@{ Arquivo = $_.Name; Bytes = $_.Length; SHA256 = (Get-FileHash -LiteralPath
$_.FullName -Algorithm SHA256).Hash }
})

$manifesto = [ordered]@{
    Caso          = $Caso
    Computador    = $env:COMPUTERNAME
    Coletor       = [Security.Principal.WindowsIdentity]::GetCurrent().Name
    Interface     = $Interface
    FiltroCaptura = $FiltroCaptura
    InicioUtc     = $inicio.ToString('o')
    FimUtc        = $fim.ToString('o')
    Dumpcap       = $Dumpcap
    Argumentos    = ($argumentos -join ' ')
    CodigoSaida   = $codigo
    SaidaDumpcap  = (($saida | Out-String).Trim())
    Arquivos      = $arquivos
}
$manifesto | ConvertTo-Json -Depth 4 | Set-Content -LiteralPath (Join-Path -Path $pasta -ChildPath
'manifesto.json') -Encoding UTF8
$arquivos | Format-Table -AutoSize
Write-Output ("Manifesto e capturas em {0}" -f $pasta)
Write-Output 'Confira na saida do dumpcap a linha de pacotes recebidos/descartados.'
if ($codigo -ne 0) { exit 1 }
exit 0
```

20 Scripts de análise e validação

Arquivo	Estado	Função
analisar_capturas.sh	EXECUTADO	Gera todas as saídas deste manual com TShark (Linux).
validar_filtros.sh + filtros.txt	EXECUTADO	Valida cada filtro de exibição e conta quadros nas 10 capturas (Apêndice B).
gerar_capturas.py	EXECUTADO	Gera as capturas sintéticas (capítulo 16).
analisar_capturas.ps1	NÃO EXECUTADO	Versão Windows das análises principais.
coletar_captura.ps1	NÃO EXECUTADO	Captura com prazo, buffer circular, hash e manifesto.
validar_sintaxe.ps1	NÃO EXECUTADO	Parser oficial do PowerShell nos .ps1; rode primeiro no Windows.

EXECUTADO Validação de sintaxe dos .ps1 (gramática tree-sitter [30]; os scripts usam só ASCII porque o Windows PowerShell 5.1 lê arquivos sem BOM como ANSI):

```
tree-sitter 0.26.0 / tree-sitter-language-pack 1.22.1 (gramatica powershell)
[OK]   analisar_capturas.ps1           56 linhas, sem nos de erro, ASCII
[OK]   coletar_captura.ps1             91 linhas, sem nos de erro, ASCII
[OK]   validar_sintaxe.ps1             30 linhas, sem nos de erro, ASCII
Resultado: 0 arquivo(s) com problema
```

EXECUTADO analisar_capturas.sh

```
#!/usr/bin/env bash
# Analise das capturas sinteticas com TShark (somente leitura: nao altera os arquivos).
# Uso: ./analisar_capturas.sh <pasta_capturas> <pasta_saidas>
# Executado nesta edicao em Linux com TShark 4.2.2; as saidas estao no manual.
set -u
C="${1:-capturas}"; O="${2:-saidas}"; mkdir -p "$O"
T() { tshark -n "$@" 2>/dev/null; }

{ echo "TShark: $(tshark --version 2>/dev/null | head -1)"; echo
  echo "SHA256 das capturas"; (cd "$C" && sha256sum *.pcapng); } > "$O/00_versao_hash.txt"

# Mesmo arquivo, dois relogios: capinfos no fuso local x TShark em UTC
{ echo '$ TZ=America/Sao_Paulo capinfos -a -e 05_lentidao_dns.pcapng'
  TZ=America/Sao_Paulo capinfos -a -e "$C/05_lentidao_dns.pcapng" 2>/dev/null | sed "s#$C/##"
  echo; echo '$ tshark -r 05_lentidao_dns.pcapng -t ud -c 2'
  T -r "$C/05_lentidao_dns.pcapng" -t ud -c 2; } > "$O/01_fuso.txt"

# Filtros de exibicao: quantos quadros cada um seleciona em 01_dns (19 quadros)
{ printf '%-38s %s\n' 'Filtro de exibicao' 'Quadros'
  for f in 'dns' 'udp.port == 53 || tcp.port == 53' 'dns.flags.response == 0' 'dns.flags.response == 1'
\
           'dns.flags.rcode != 0' 'dns.flags.rcode == 3' 'dns.flags.truncated == 1' 'tcp.port == 53
&& !dns'; do
    printf '%-38s %s\n' "$f" "$(T -r "$C/01_dns.pcapng" -Y "$f" | wc -l)"; done; } > "$O/
02_filtros_dns.txt"

{ echo 'Respostas DNS (campos):'
  T -r "$C/01_dns.pcapng" -Y 'dns.flags.response == 1' -T fields -E header=y -E separator=' | ' \
    -e frame.number -e dns.response_to -e dns.qry.name -e dns.qry.type -e dns.flags.rcode -e
dns.flags.truncated -e dns.time -e dns.a -e dns.resp.ttl
  echo; echo 'Conversa TCP/53 depois da resposta truncada:'
  T -r "$C/01_dns.pcapng" -Y 'tcp.port == 53'; } > "$O/03_dns.txt"
```

```
{ echo '02_tcp_ok (handshake, SMB2 Negotiate e encerramento):'; T -r "$C/02_tcp_ok.pcapng"
  echo; echo 'iRTT calculado pelo Wireshark:'; T -r "$C/02_tcp_ok.pcapng" -Y 'tcp.analysis.initial_rtt'
-T fields -e frame.number -e tcp.analysis.initial_rtt | head -1
  echo; echo '03_tcp_rst:'; T -r "$C/03_tcp_rst.pcapng"
  echo; echo '04_syn_sem_resposta:'; T -r "$C/04_syn_sem_resposta.pcapng"; } > "$O/04_tcp.txt"

{ echo 'Caso A - marcos de tempo (05_lentidao_dns):'; T -r "$C/05_lentidao_dns.pcapng"
  echo; echo 'Medidas do proprio Wireshark:'
  printf 'dns.time                %s\n' "$(T -r "$C/05_lentidao_dns.pcapng" -Y 'dns.time' -T fields -e
dns.time)"
  printf 'tcp.analysis.initial_rtt %s\n' "$(T -r "$C/05_lentidao_dns.pcapng" -Y
'tcp.analysis.initial_rtt' -T fields -e tcp.analysis.initial_rtt | head -1)"
  printf 'http.time               %s\n' "$(T -r "$C/05_lentidao_dns.pcapng" -Y 'http.time' -T fields -
e http.time)"
  printf 'Filtro dns.time > 1     %s quadro(s)\n' "$(T -r "$C/05_lentidao_dns.pcapng" -Y 'dns.time >
1' | wc -l)"; } > "$O/05_caso_a.txt"

{ echo '06_retransmissao (captura no cliente, desde o SYN):'; T -r "$C/06_retransmissao.pcapng"
  echo; printf '%-36s %s\n' 'Filtro' 'Quadros'
  for f in tcp.analysis.retransmission tcp.analysis.fast_retransmission tcp.analysis.duplicate_ack \
           tcp.analysis.zero_window tcp.analysis.zero_window_probe tcp.analysis.window_update; do
    printf '%-36s %s\n' "$f" "$(T -r "$C/06_retransmissao.pcapng" -Y "$f" | wc -l)"; done
  echo; echo 'Expert Information (resumo):'
  T -r "$C/06_retransmissao.pcapng" -q -z expert,note | sed '/^$/d' | head -30; } > "$O/
06_retransmissao.txt"

{ echo '10_meio_da_sessao (mesma conversa, captura iniciada depois do handshake):'
  T -r "$C/10_meio_da_sessao.pcapng" | head -8
  echo; printf 'SYN presentes: %s\n' "$(T -r "$C/10_meio_da_sessao.pcapng" -Y 'tcp.flags.syn == 1' | wc
-l)"
  printf 'iRTT disponivel: %s\n' "$(T -r "$C/10_meio_da_sessao.pcapng" -Y 'tcp.analysis.initial_rtt' |
wc -l)"; } > "$O/07_meio_sessao.txt"

{ echo '07_smb_acesso_negado:'; T -r "$C/07_smb_acesso_negado.pcapng"
  echo; echo 'Respostas SMB2 com erro:'
  T -r "$C/07_smb_acesso_negado.pcapng" -Y 'smb2.nt_status != 0' -T fields -E separator=' | ' \
    -e frame.number -e smb2.cmd -e smb2.nt_status -e smb2.msg_id
  printf 'Filtro smb2.nt_status == 0xc0000022: %s quadro(s)\n' "$(T -r "$C/07_smb_acesso_negado.pcapng"
-Y 'smb2.nt_status == 0xc0000022' | wc -l)"
  echo; echo 'Hierarquia de protocolos:'; T -r "$C/07_smb_acesso_negado.pcapng" -q -z io,phs | sed '/
^=*$/d'
  echo; echo 'Conversas TCP:'; T -r "$C/07_smb_acesso_negado.pcapng" -q -z conv,tcp | sed '/^=*$/d'; }
> "$O/08_smb.txt"

{ echo '08_kerberos_preauth:'; T -r "$C/08_kerberos_preauth.pcapng"
  echo; T -r "$C/08_kerberos_preauth.pcapng" -Y 'kerberos.error_code' -T fields -E header=y -E
separator=' | ' \
    -e frame.number -e kerberos.error_code -e kerberos.CNameString -e kerberos.realm
  printf 'Filtro kerberos.error_code == 24: %s quadro(s)\n' "$(T -r "$C/08_kerberos_preauth.pcapng" -Y
'kerberos.error_code == 24' | wc -l)"; } > "$O/09_kerberos.txt"

{ echo '09_tls13:'; T -r "$C/09_tls13.pcapng"
  echo; T -r "$C/09_tls13.pcapng" -Y 'tls.handshake.type' -T fields -E header=y -E separator=' | ' \
    -e frame.number -e tls.record.version -e tls.handshake.type -e tls.handshake.extensions_server_name
-e tls.handshake.extensions.supported_version
  printf 'Quadros com certificado visivel (tls.handshake.type == 11): %s\n' "$(T -r "$C/
09_tls13.pcapng" -Y 'tls.handshake.type == 11' | wc -l)"; } > "$O/10_tls.txt"
echo "Saidas em $O"
```

POWERSHELL NÃO EXECUTADO analisar_capturas.ps1

```
<#
.SYNOPSIS
  Versao Windows de analisar_capturas.sh: aplica os filtros do manual com TShark (somente leitura).
```

```
.DESCRIPTION
  Le as capturas da pasta e grava um .txt por analise na pasta de saida.
  ESTADO NESTA EDICAO: NAO EXECUTADO no Windows (apenas validacao de sintaxe).
  A versao .sh, com os mesmos filtros, foi executada em Linux (TShark 4.2.2).
.EXAMPLE
  .\analisar_capturas.ps1 -Capturas .\capturas -Saidas .\saidas
#>
[CmdletBinding()]
param(
    [string]$Capturas = (Join-Path -Path $PSScriptRoot -ChildPath 'capturas'),
    [string]$Saidas = (Join-Path -Path $PSScriptRoot -ChildPath 'saidas'),
    [string]$TShark = (Join-Path -Path $env:ProgramFiles -ChildPath 'Wireshark\tshark.exe')
)

if (-not (Test-Path -LiteralPath $TShark)) { Write-Error "tshark nao encontrado em $TShark"; exit 3 }
New-Item -ItemType Directory -Path $Saidas -Force | Out-Null

function Invoke-TShark {
    param([string[]]$Argumentos)
    & $TShark @(@('-n') + $Argumentos) 2>$null
}

function Get-Contagem {
    param([string]$Arquivo, [string]$Filtro)
    @(Invoke-TShark -Argumentos @('-r', (Join-Path -Path $Capturas -ChildPath $Arquivo), '-Y',
$Filtro)).Count
}

$tabelas = @(
    @{ Saida = '02_filtros_dns.txt'; Arquivo = '01_dns.pcapng'; Filtros = @('dns', 'udp.port == 53 ||
tcp.port == 53', 'dns.flags.response == 0',
        'dns.flags.response == 1', 'dns.flags.rcode != 0', 'dns.flags.rcode == 3', 'dns.flags.truncated
== 1', 'tcp.port == 53 && !dns') },
    @{ Saida = '06_filtros_tcp.txt'; Arquivo = '06_retransmissao.pcapng'; Filtros =
@('tcp.analysis.retransmission', 'tcp.analysis.fast_retransmission',
        'tcp.analysis.duplicate_ack', 'tcp.analysis.zero_window', 'tcp.analysis.zero_window_probe',
'tcp.analysis.window_update') },
    @{ Saida = '12_filtros_dominio.txt'; Arquivo = '07_smb_acesso_negado.pcapng'; Filtros = @('smb2',
'smb2.nt_status != 0', 'smb2.nt_status == 0xc0000022', 'dns') }
)

foreach ($t in $tabelas) {
    $linhas = foreach ($f in $t.Filtros) { '{0,-38} {1}' -f $f, (Get-Contagem -Arquivo $t.Arquivo -
Filtro $f) }
    @(('Arquivo: {0}' -f $t.Arquivo), ('{0,-38} {1}' -f 'Filtro de exibicao', 'Quadros')) + $linhas |
        Set-Content -LiteralPath (Join-Path -Path $Saidas -ChildPath $t.Saida) -Encoding UTF8
}

$campos = @('-T', 'fields', '-E', 'header=y', '-E', 'separator=|')
$dns = Invoke-TShark -Argumentos (@('-r', (Join-Path -Path $Capturas -ChildPath '01_dns.pcapng'), '-Y',
'dns.flags.response == 1') + $campos +
    @('-e', 'frame.number', '-e', 'dns.response_to', '-e', 'dns.qry.name', '-e', 'dns.flags.rcode', '-
e', 'dns.flags.truncated', '-e', 'dns.time'))
$dns | Set-Content -LiteralPath (Join-Path -Path $Saidas -ChildPath '03_dns_campos.txt') -Encoding UTF8

$utc = Invoke-TShark -Argumentos @('-r', (Join-Path -Path $Capturas -ChildPath
'05_lentidao_dns.pcapng'), '-t', 'ud', '-c', '2')
$utc | Set-Content -LiteralPath (Join-Path -Path $Saidas -ChildPath '01_utc.txt') -Encoding UTF8

Get-ChildItem -LiteralPath $Saidas -Filter '*.txt' | Sort-Object Name | ForEach-Object {
    Write-Output ('##### {0}' -f $_.Name)
    Get-Content -LiteralPath $_.FullName
}
```

21 Casos de diagnóstico resolvidos

Caso A — lentidão antes da conexão (`05_lentidao_dns`)

EXECUTADO SINTÉTICO

```
Caso A - marcos de tempo (05_lentidao_dns):
    1   0.000000  10.10.10.20 → 10.10.10.10  DNS 75 Standard query 0x2b01 A app.ad.lab.test
    2   2.000000  10.10.10.10 → 10.10.10.20  DNS 106 Standard query response 0x2b01 A app.ad.lab.test A
10.10.10.40
    3   2.005000  10.10.10.20 → 10.10.10.40  TCP 54 51030 → 80 [SYN] Seq=0 Win=64240 Len=0
    4   2.010000  10.10.10.40 → 10.10.10.20  TCP 54 80 → 51030 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0
    5   2.011000  10.10.10.20 → 10.10.10.40  TCP 54 51030 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
    6   2.012000  10.10.10.20 → 10.10.10.40  HTTP 121 GET /relatorio HTTP/1.1
    7   2.050000  10.10.10.40 → 10.10.10.20  HTTP 131 HTTP/1.1 200 OK  (text/plain)
    8   2.051000  10.10.10.20 → 10.10.10.40  TCP 54 51030 → 80 [ACK] Seq=68 Ack=78 Win=64240 Len=0

Medidas do proprio Wireshark:
dns.time                 2.000000000
tcp.analysis.initial_rtt 0.006000000
http.time                0.038000000
Filtro dns.time > 1      1 quadro(s)
```

As três medidas do próprio Wireshark separam o tempo: 2 s entre consulta e resposta DNS (`dns.time`), 6 ms de handshake (iRTT) e 38 ms entre o pedido HTTP e a resposta (`http.time`). O usuário esperou cerca de 2,05 s, e quase tudo foi antes da conexão existir. Investigue o resolvedor, a rede até ele e as dependências dele; não atribua os 2 s automaticamente ao processamento do DNS, e muito menos à aplicação.

Caso B — SYNs sem retorno (`04_syn_sem_resposta`)

O cliente envia SYN em 0, 1 e 3 s e não há resposta na captura (capítulo 10). Conclusão: tentativas sem retorno observadas naquele ponto. Verifique rota, destino, firewall, caminho de volta e qualidade da captura. Para atribuir ao servidor, obtenha evidência do outro lado.

Caso C — transporte funciona, acesso é negado (`07_smb_acesso_negado`)

Handshake completo, sessão e conexão ao compartilhamento com sucesso, e `smb2.nt_status == 0xc0000022` no Create (capítulo 14). Investigue identidade, grupos, permissões de compartilhamento e NTFS. Abrir portas não corrige autorização quando o transporte já foi demonstrado.

Caso D — retransmissão sem contexto (`10_meio_da_sessao`)

EXECUTADO SINTÉTICO A mesma conversa de `06`, com a captura começando depois do handshake:

```
10_meio_da_sessao (mesma conversa, captura iniciada depois do handshake):
    1   0.000000  10.10.10.20 → 10.10.10.30  TCP 1054 51040 → 40000 [PSH, ACK] Seq=1 Ack=1 Win=64240
Len=1000 [TCP segment of a reassembled PDU]
    2   0.001000  10.10.10.20 → 10.10.10.30  TCP 1054 51040 → 40000 [PSH, ACK] Seq=1001 Ack=1 Win=64240
Len=1000 [TCP segment of a reassembled PDU]
    3   0.004000  10.10.10.30 → 10.10.10.20  TCP 54 40000 → 51040 [ACK] Seq=1 Ack=4294966297 Win=64240
Len=0
    4   0.006000  10.10.10.30 → 10.10.10.20  TCP 54 [TCP Dup ACK 3#1] 40000 → 51040 [ACK] Seq=1
Ack=4294966297 Win=64240 Len=0
    5   0.008000  10.10.10.30 → 10.10.10.20  TCP 54 [TCP Dup ACK 3#2] 40000 → 51040 [ACK] Seq=1
Ack=4294966297 Win=64240 Len=0
    6   0.010000  10.10.10.30 → 10.10.10.20  TCP 54 [TCP Dup ACK 3#3] 40000 → 51040 [ACK] Seq=1
Ack=4294966297 Win=64240 Len=0
    7   0.012000  10.10.10.20 → 10.10.10.30  TCP 1054 [TCP Fast Retransmission] 51040 → 40000 [PSH,
ACK] Seq=4294966297 Ack=1 Win=64240 Len=1000 [TCP segment of a reassembled PDU]
    8   0.014000  10.10.10.30 → 10.10.10.20  TCP 54 40000 → 51040 [ACK] Seq=1 Ack=2001 Win=64240 Len=0

SYN presentes: 0
iRTT disponivel: 0
```

Sem o SYN, o Wireshark não tem o número de sequência inicial: os números relativos ficam distorcidos (o ACK aparece como 4294966297) e não há iRTT. A retransmissão ainda é marcada, mas não há base para atribuir a lentidão à rede. Refaça a coleta desde o início ou compare pontos; avalie sequência, perdas e duplicação de espelhamento.

22 Caso integrado com perguntas

SINTÉTICO ana.lab informa que um arquivo não abre em FS01. Há a captura `07_smb_acesso_negado` feita em CL01 (14:00:00–14:00:01 UTC) e os logs do servidor.

Evidência	Observação
Quadros 1–2	DNS A de `fs01.ad.lab.test` responde 10.10.10.30, RCODE 0, em 2 ms.
Quadros 3–5	SYN, SYN/ACK e ACK para TCP/445.
Quadros 6–11	Negociação SMB2, Session Setup e Tree Connect em `\\fs01.ad.lab.test\LabShare`, todos com sucesso.
Quadros 12–13	Create de `restrito.txt` respondido com STATUS_ACCESS_DENIED (0xc0000022).
FS01 Security	4624 tipo 3 de ana.lab e evidência de negação pertinente ao recurso (capítulo 15).
Cliente	Mensagem "Acesso negado"; outros arquivos do mesmo compartilhamento abrem.

Perguntas e gabarito

01 **O DNS falhou?** Não há evidência de falha nessa resolução: RCODE 0 em 2 ms.

02 **A porta estava inacessível?** O handshake observado contraria essa hipótese para esta conexão.

03 **A autenticação falhou?** O Session Setup teve sucesso. Autenticação, porém, não garante autorização.

04 **A causa é a permissão NTFS?** Ainda não se sabe. A conexão ao compartilhamento passou, o que sugere negação no arquivo, mas é preciso verificar permissões efetivas e o log do recurso.

CONCLUSÃO Resolução e conectividade observadas avançaram até a operação da aplicação, que foi negada com STATUS_ACCESS_DENIED na abertura do arquivo. Os dados apontam para investigação de autorização. Não afirmar que "a rede inteira está normal" nem que a causa é uma ACL específica sem verificá-la.

A Formulários

A.1 Checklist de captura

☐	Pergunta e janela temporal (UTC) definidas
☐	Ponto de captura e interface identificados; limites anotados
☐	Relógio do coletor conferido; exibição em UTC
☐	Resolução de nomes de rede desligada
☐	Filtro de captura anotado (ou nenhum)
☐	Prazo e buffer circular definidos (`-a` , `-b`)
☐	Ação reproduzida com horário de início e fim
☐	Perda reportada e snaplen verificados
☐	Arquivo original preservado e SHA256 calculado

A.2 Relatório de evidência

Campo	Preenchimento
Caso / pergunta	
Hosts / interfaces / ponto de captura	
Início e fim UTC / ação reproduzida	
Filtros / snaplen / perda reportada	
Arquivo original / SHA256	
Quadros / streams / eventos correlacionados	
Fatos observados	
Hipóteses e testes	
Conclusão e limites	
Próxima ação / responsável	

B Referência de filtros validados

Todos os filtros de exibição citados nesta apostila, validados no TShark 4.2.2 e aplicados às 10 capturas sintéticas. "Quadros" é a soma nos dez arquivos; zero significa que o filtro é válido mas nenhuma captura tem aquele protocolo ou condição (por exemplo, não há IPv6, ICMP, QUIC nem NTLM nas amostras). O controle negativo prova que o validador rejeita um campo inexistente.

EXECUTADO `./validar_filtros.sh filtros.txt capturas`

```
TShark: TShark (Wireshark) 4.2.2 (Git v4.2.2 packaged as 4.2.2-1.1build3).
Filtro                                              Sintaxe Quadros (10 capturas)
ip.addr == 10.10.10.20                              OK     97
ipv6.addr == 2001:db8::20                           OK     0
tcp.port == 445                                     OK     24
udp.port == 53 || tcp.port == 53                    OK     23
ip.addr == 10.10.10.20 && dns                       OK     16
ip.addr == 10.10.10.0/24                            OK     97
ip.addr == 10.10.10.20 && tcp.port == 445           OK     24
ip.src == 10.10.10.20 && ip.dst == 10.10.10.30      OK     31
!(ip.addr == 10.10.10.20)                           OK     0
tcp.stream == 0                                     OK     81
arp || icmpv6                                       OK     0
dns                                                 OK     16
dns.flags.response == 0                             OK     8
dns.flags.response == 1                             OK     8
dns.qry.name == "fs01.ad.lab.test"                  OK     4
dns.flags.rcode != 0                                OK     2
dns.flags.rcode == 3                                OK     1
dns.flags.truncated == 1                            OK     1
dns.time > 1                                        OK     1
tcp.port == 53 && !dns                              OK     7
tcp.flags.syn == 1 && tcp.flags.ack == 0            OK     10
tcp.flags.syn == 1 && tcp.flags.ack == 1            OK     6
tcp.flags.reset == 1                                OK     1
tcp.flags.fin == 1                                  OK     4
tcp.flags.syn == 1 || tcp.flags.reset == 1          OK     17
icmp || icmpv6                                      OK     0
tcp.analysis.retransmission                         OK     4
tcp.analysis.fast_retransmission                    OK     2
tcp.analysis.duplicate_ack                          OK     6
tcp.analysis.out_of_order                           OK     0
tcp.analysis.zero_window                            OK     4
tcp.analysis.zero_window_probe                      OK     2
tcp.analysis.window_update                          OK     2
tcp.analysis.initial_rtt                            OK     50
tcp.analysis.flags                                  OK     18
http.request || http.response                       OK     2
http.time > 0.5                                     OK     0
tls                                                 OK     5
tls.handshake.type == 1                             OK     1
tls.handshake.type == 2                             OK     1
tls.handshake.type == 11                            OK     0
tls.handshake.extensions_server_name == "app.ad.lab.test" OK     1
quic                                                OK     0
udp.port == 443                                     OK     0
tls || quic                                         OK     5
dns || kerberos || ldap || smb2 || dcerpc           OK     28
kerberos || ldap || smb2                            OK     12
kerberos.error_code == 24                           OK     1
cldap                                               OK     0
ntlmssp                                             OK     0
smb2                                                OK     10
smb2.nt_status != 0                                 OK     1
```

```
smb2.nt_status == 0xc0000022                          OK     1
smb2.cmd == 5                                         OK     2
Filtros com erro: 0
Controle negativo 'tcp.flagz == 1': tshark: Constant expression is invalid.
    tcp.flagz == 1
    ^~~~~~~~~~~~~
```

C Glossário

Termo	Significado
Frame / quadro	Unidade registrada na captura.
Stream	Conversa identificada pelo analisador; o índice vale só naquela captura.
Dissector	Módulo que interpreta um protocolo.
Snaplen	Limite de bytes capturados por pacote.
iRTT / RTT	Tempo de ida e volta: o inicial medido no handshake e o de cada confirmação.
pcapng	Formato com metadados e várias interfaces.
BPF	Linguagem dos filtros de captura.
SPAN / TAP	Espelhamento configurado no switch / dispositivo que copia o enlace.
Offload	Processamento (checksum, segmentação) transferido para a placa.
Buffer circular	Série de arquivos rotativos em que os mais antigos são apagados.
RCODE	Código de resposta DNS (0 sucesso, 2 SERVFAIL, 3 NXDOMAIN).
SNI	Nome do servidor enviado em claro no ClientHello, salvo com ECH.
NT Status	Código de resultado do SMB2 (ex.: 0xc0000022, acesso negado).
Captura sintética	Arquivo de captura cujos pacotes foram montados por script, sem tráfego real.

ANTES DE CONCLUIR A captura contém os dois sentidos? Começou antes da ação? O horário está em UTC? Há tráfego cifrado? O filtro esconde contexto? O ponto representa o caminho investigado? Os logs confirmam o resultado da aplicação?

D Fontes oficiais

Consultadas em 29/10/2026. Menus e campos do Wireshark variam entre versões; esta edição usa o TShark 4.2.2 como referência.

[1] Wireshark User's Guide
https://www.wireshark.org/docs/wsug_html/

[2] Wireshark Wiki — CaptureSetup
https://wiki.wireshark.org/CaptureSetup

[3] Wireshark Wiki — Offloading
https://wiki.wireshark.org/CaptureSetup/Offloading

[4] Wireshark Wiki — CaptureFilters
https://wiki.wireshark.org/CaptureFilters

[5] wireshark-filter (sintaxe de exibição)
https://www.wireshark.org/docs/man-pages/wireshark-filter.html

[6] RFC 1035 — DNS
https://www.rfc-editor.org/rfc/rfc1035.html

[7] RFC 7766 — DNS sobre TCP
https://www.rfc-editor.org/rfc/rfc7766.html

[8] RFC 9293 — TCP
https://www.rfc-editor.org/rfc/rfc9293.html

[9] WSUG — TCP Analysis
https://www.wireshark.org/docs/wsug_html_chunked/ChAdvTCPAnalysis.html

[10] WSUG — Expert Information
https://www.wireshark.org/docs/wsug_html_chunked/ChAdvExpert.html

[11] WSUG — Following Streams
https://www.wireshark.org/docs/wsug_html_chunked/ChAdvFollowStreamSection.html

[12] Wireshark Wiki — TLS
https://wiki.wireshark.org/TLS

[13] RFC 8446 — TLS 1.3
https://www.rfc-editor.org/rfc/rfc8446.html
Mensagens após o ServerHello cifradas; versão legada no registro.

[14] Microsoft — Configure firewall for AD domains and trusts
https://learn.microsoft.com/en-us/troubleshoot/windows-server/identity/config-firewall-for-ad-domains-and-trusts

[15] Microsoft — Service overview and network port requirements
https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/service-overview-and-network-port-requirements
464, 3268/3269, faixa dinâmica 49152–65535.

[16] Microsoft — Evento 4769
https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-4769

[17] Microsoft — SMB security
https://learn.microsoft.com/en-us/windows-server/storage/file-server/smb-security

[18] dumpcap
https://www.wireshark.org/docs/man-pages/dumpcap.html

[19] TShark
https://www.wireshark.org/docs/man-pages/tshark.html

[20] WSUG — Time Stamps
https://www.wireshark.org/docs/wsug_html_chunked/ChAdvTimestamps.html

[21] WSUG — Time Zones
https://www.wireshark.org/docs/wsug_html_chunked/ChAdvTimezones.html

[22] WSUG — Name Resolution
https://www.wireshark.org/docs/wsug_html_chunked/ChAdvNameResolutionSection.html

[23] WSUG — The View menu
https://www.wireshark.org/docs/wsug_html_chunked/ChUseViewMenuSection.html

[24] Display Filter Reference — DNS
https://www.wireshark.org/docs/dfref/d/dns.html
dns.time, dns.response_to, dns.flags.rcode.

[25] Microsoft — Test-NetConnection
https://learn.microsoft.com/en-us/powershell/module/nettcpip/test-netconnection

[26] Microsoft — MS-ERREF NTSTATUS Values
https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-erref/596a1078-e883-4972-9bbc-49e60bebca55

[27] Microsoft — Evento 4771
https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-4771
0x18 = KDC_ERR_PREAUTH_FAILED.

[28] capinfos
https://www.wireshark.org/docs/man-pages/capinfos.html

[29] Scapy
https://pypi.org/project/scapy/
Versão 2.8.0, usada para gerar as capturas sintéticas.

[30] tree-sitter-language-pack
https://pypi.org/project/tree-sitter-language-pack/
Ferramenta de terceiros, só para validar a sintaxe dos .ps1.

E Registro de edições e verificação

Edição	Data	Conteúdo
Edição 1	29/10/2026	21 seções: captura, filtros, DNS, TCP, TLS, domínio, correlação, laboratórios, casos e fontes. Sem execução.
Edição 2	29/10/2026	Padrão da série; capítulos de tempo/fuso e de captura com prazo; 10 capturas sintéticas anexadas; saídas reais de TShark; filtros validados; correções abaixo.

E.1 Correções e acréscimos em relação à Edição 1

Ponto	Mudança
Certificado no TLS 1.3	"Não fica necessariamente visível" passou a "vai sempre cifrado", conforme a RFC 8446 [13].
Horário exibido	Novo capítulo 4: o Wireshark exibe hora local; demonstração executada com o mesmo arquivo em dois fusos.
Resolução de nomes	Recomendação de desligar, para o Wireshark não gerar DNS na própria captura [22].
Portas do domínio	Acrescentados 464, 3268/3269 e a faixa dinâmica de RPC 49152–65535 [15].
Filtros de diagnóstico	dns.time, smb2.nt_status, kerberos.error_code, ntlmssp, cldap, tcp.analysis.initial_rtt, http.time.
Test-NetConnection	Aviso de que o teste também gera ping e resolução de nome [25].
Coleta longa	Buffer circular do dumpcap, executado de verdade em loopback.
Forma	Capa, sumário, apêndices, fontes embutidas (acentos nos blocos), anexo com laboratório.

E.2 Verificação desta edição

- 54 filtros de exibição e 5 de captura validados no TShark/dumpcap 4.2.2, com controles negativos rejeitados.
- Capturas sem quadros malformados nem erros de dissector; geração reprodutível (mesmos SHA256 em duas execuções).
- Saídas dos capítulos 4, 7 a 14, 16, 19 e 21 copiadas da execução real de `analisar_capturas.sh`.
- Sintaxe dos .ps1 validada com tree-sitter; nada executado no Windows.

E.3 Pendências

- Executar os laboratórios dos capítulos 17 e 18 num domínio de teste e trocar as marcas para EXECUTADO.
- Rodar `validar_sintaxe.ps1`, `coletar_captura.ps1` e `analisar_capturas.ps1` no Windows com Npcap.
- Conferir no Wireshark gráfico da versão do laboratório os rótulos dos menus citados.